Graph Structure Learning for Traffic Prediction

Mahmood Amintoosi[1*]

[1*]Division of Computer Science, Department of Applied Mathematics, Faculty of Mathematical Sciences, Ferdowsi University of Mashhad, Bahonar Av., Mashhad, 91775, Khorasan-e Razavi, IRAN, ORCID:0000-0001-9640-6475.

Corresponding author(s). E-mail(s): m.amintoosi@um.ac.ir;

Abstract

Graph-based traffic forecasting methods typically rely on predefined adjacency matrices derived from physical road proximity. However, dense physical connectivity can induce excessive spatial aggregation—a form of oversmoothing—that degrades prediction accuracy. We propose a multi-lag graph structure learning framework that discovers temporally heterogeneous functional dependencies between traffic sensors. Using DAGMA continuous optimization, we construct lag-specific dependency graphs from explicit temporal input blockings of the form $\boldsymbol{Z} = [\boldsymbol{x(t-L)}, \ldots, \boldsymbol{x(t)}]$, yielding separate functional graphs for each temporal lag. A T-GCN-MultiGSL-Mix architecture then combines these lag-specific graphs through a per-node, per-timestep learned mixing gate. Experiments on the Los-loop highway dataset demonstrate a mean RMSE improvement of 14.9% over a no-graph baseline across five random seeds, with the improvement robust to a parameter-matched control. A lag ablation study confirms that all three temporal lags contribute complementary information. On the SZ-Taxi urban dataset, improvements are marginal, indicating dataset dependence. These findings establish that temporally heterogeneous graph structures can substantially benefit traffic forecasting when the underlying dependency patterns vary across temporal scales.

Keywords: Traffic Forecasting, Graph Structure Learning, Multi-Lag Dependency Learning, Graph Convolutional Networks, Learned Graph Mixing

1 Introduction

Traffic prediction is a fundamental challenge in intelligent transportation systems, requiring models that capture complex spatial and temporal dependencies in urban road networks [**?**]. Graph Convolutional Networks (GCNs) [**?**] and Temporal GCNs (T-GCNs) [**?**] have emerged as effective approaches by modeling traffic networks as graphs, where nodes represent road segments and edges encode spatial relationships.

A critical limitation of existing graph-based methods is their reliance on predefined adjacency matrices derived from physical road proximity [**?**]. A systematic bibliometric analysis of 784 traffic forecasting articles (2000–2025) confirms that graph-based methods now dominate the field, yet nearly all depend on heuristic graph construction from road network topology (see Appendix A). However, physical connectivity is an imperfect proxy for functional traffic dependency: roads that are physically distant may exhibit strong traffic correlations due to commuting patterns, while adjacent roads may have minimal mutual influence [**?**].

A second, less recognized problem is that dense physical graphs can induce *over-smoothing* in GCN-based models. When the adjacency matrix encodes hundreds or thousands of edges, repeated graph convolution operations aggregate information from increasingly distant nodes, causing node representations to converge and lose discriminative power. This excessive spatial aggregation can paradoxically degrade forecasting performance compared to using no graph at all.

Graph Structure Learning (GSL) addresses the first problem by learning the adjacency matrix directly from traffic data rather than relying on physical proximity. Recent work has applied continuous DAG optimization methods such as NOTEARS [**?**] and DAGMA [**?**] to this task, discovering sparse functional dependencies that differ substantially from physical road connectivity [**?**].

However, a single static learned graph has an inherent limitation: traffic dependencies are *temporally heterogeneous*. The influence of sensor i on sensor j may vary depending on the temporal lag—a 5-minute delay captures different traffic propagation patterns than a 15-minute delay. A single adjacency matrix cannot represent these lag-specific structures.

In this paper, we address this limitation through two contributions:

1. **Multi-lag DAGMA:** We construct explicit temporal input blockings $Z = [x(t-L), \ldots, x(t)]$ and apply DAGMA to learn a weight matrix whose blocks correspond to lag-specific temporal functional dependencies. This provides direct, interpretable evidence for which temporal lags carry predictive information.
2. **T-GCN-MultiGSL-Mix:** We propose a T-GCN variant that processes each lag-specific graph separately and uses a per-node, per-timestep learned gate to combine the resulting representations through learned mixing.

Experiments on the Los-loop highway dataset demonstrate that T-GCN-MultiGSL-Mix achieves a mean RMSE improvement of 14.9% over a no-graph baseline, validated across five random seeds and confirmed against a parameter-matched control. A lag ablation study shows that all three temporal lags contribute complementary

information. On the SZ-Taxi urban dataset, improvements are marginal, and we explicitly report this dataset dependence rather than claiming universal benefit.

The remainder of this paper is organized as follows. Section 2 reviews background on GCNs and T-GCNs. Section 3 presents the multi-lag DAGMA formulation and T-GCN-MultiGSL-Mix architecture. Section 4 describes the experimental setup. Section 5 presents results. Section 6 discusses findings. Section 7 states limitations. Section 8 concludes.

2 Background

2.1 Problem Formulation

We model the road network as a graph $G = (V, E)$, where $V = \{v_1, \ldots, v_N\}$ represents N road segments and $\mathbf{A} \in \{0, 1\}^{N \times N}$ is the binary adjacency matrix encoding road connectivity. Traffic data is represented by a feature matrix $X \in \mathbb{R}^{T \times N}$, where $X_t \in \mathbb{R}^N$ represents traffic speeds across all roads at time t. The forecasting task learns a mapping from historical observations to future traffic conditions:

$$[X_{t+1}, \ldots, X_{t+T}] = f(G; [X_{t-n}, \ldots, X_t]) \tag{2.1}$$

where n is the historical window size and T is the prediction horizon.

2.2 Graph Convolutional Networks

GCNs extend convolution to graph-structured data through a layer-wise propagation rule [**?**]:

$$\mathbf{H}^{(l+1)} = \sigma\Big(\tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}}\mathbf{H}^{(l)}\mathbf{W}^{(l)}\Big) \tag{2.2}$$

where $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}_N$ is the adjacency matrix with self-connections, $\tilde{\mathbf{D}}$ is the degree matrix, $\mathbf{W}^{(l)}$ is a trainable weight matrix, and $\sigma(\cdot)$ is a nonlinear activation. The normalized adjacency $\hat{A} = \tilde{\mathbf{D}}^{-\frac{1}{2}}\tilde{\mathbf{A}}\tilde{\mathbf{D}}^{-\frac{1}{2}}$ aggregates information from neighboring nodes while preserving feature scale. A two-layer GCN is:

$$f(\mathbf{A}, \mathbf{X}) = \sigma\Big(\hat{A}\sigma\Big(\hat{A}\mathbf{X}\mathbf{W}^{(1)}\Big)\mathbf{W}^{(2)}\Big) \tag{2.3}$$

2.3 Temporal Graph Convolutional Network

The T-GCN [**?**] combines GCN for spatial feature extraction with a Gated Recurrent Unit (GRU) for temporal modeling. At each timestep, the GCN extracts spatial features $\mathbf{Z}_t = f(\mathbf{A}, \mathbf{X}_t)$, which are then processed by the GRU to capture temporal dynamics. The GRU uses update gate $\mathbf{u}_t$, reset gate $\mathbf{r}_t$, and candidate hidden state $\mathbf{c}_t$ with learnable weight matrices $\mathbf{W}_u$, $\mathbf{W}_r$, $\mathbf{W}_c$. The model is trained by minimizing:

$$\mathcal{L} = |\, Y_t - \widehat{Y}_t \,|_2 + \lambda\, |\, \Theta \,|_1 \tag{2.4}$$

where $\widehat{Y}_t$ is the prediction, Y_t is the ground truth, and λ controls L1 regularization.

3 Proposed Method

3.1 Motivation for Data-Driven Graph Structure Learning

In both GCN [**?**] and T-GCN [**?**], the adjacency matrix $\mathbf{A}$ is predefined based on physical road proximity. This has two fundamental limitations. First, physical connectivity is an imperfect proxy for functional traffic dependency: roads that are physically distant may exhibit strong traffic correlations due to commuting patterns, while adjacent roads may have minimal mutual influence. Second, dense physical graphs (e.g., 2833 edges for 207 nodes in the Los-loop dataset) can induce oversmoothing through excessive spatial aggregation in graph convolution layers.

Graph Structure Learning addresses these limitations by learning $\mathbf{A}$ directly from traffic data. We employ DAGMA [**?**], which extends the NOTEARS framework [**?**] for continuous DAG optimization.

3.2 DAGMA-Based Graph Structure Learning

DAGMA models the data through a structural equation model defined by a weighted adjacency matrix $W \in \mathbb{R}^{d\times d}$. Instead of operating on the discrete space of DAGs, it optimizes over continuous real matrices with a smooth acyclicity constraint $h(W) = 0$, where:

$$h(W) = \operatorname{tr}\left(e^{W\circ W}\right) - d = 0 \tag{3.1}$$

and $\circ$ denotes the Hadamard product. The optimization uses an augmented Lagrangian:

$$L^{\rho}(W,\alpha) = \text{score}(W) + \frac{\rho}{2}|h(W)|^2 + \alpha h(W) \tag{3.2}$$

where score(W) measures data fit, α is the Lagrange multiplier, and $\rho > 0$ is a penalty parameter. The algorithm iteratively updates W until $h(W) < \epsilon$, then applies thresholding: $\hat{W} = W \circ \mathbf{1}(|W| > \omega)$.

Convention: Throughout this paper, we follow the DAGMA convention where $W[i, j]$ represents a directed dependency from variable i to variable j.

3.3 Multi-Lag DAGMA Formulation

A single learned graph captures contemporaneous or aggregated dependencies but cannot distinguish between different temporal scales of traffic propagation. To address this, we construct an explicit multi-lag input representation.

Given traffic observations $x(t) \in \mathbb{R}^N$ and a lag window L, we define the multi-lag input:

$$Z_t = \left[x(t{-}L),\ x(t{-}L{+}1),\ \ldots,\ x(t{-}1),\ x(t)\right] \in \mathbb{R}^{(L+1)\times N} \tag{3.3}$$

which is flattened to a vector of dimension $(L + 1) \cdot N$. The matrix Z is formed by stacking Z_t over all training time steps.

Applying DAGMA to Z produces a weight matrix $W \in \mathbb{R}^{(L+1)N\times(L+1)N}$. The block structure of W is:

- $W[l \cdot N : (l{+}1) \cdot N,\ L \cdot N : (L{+}1) \cdot N]$ represents dependencies from lag l to the current time step

- $W[L \cdot N : (L{+}1) \cdot N,\ L \cdot N : (L{+}1) \cdot N]$ represents contemporaneous dependencies

Since $W[i, j]$ means variable $i \rightarrow$ variable j, the block $W[l \cdot N + i,\ L \cdot N + j]$ captures the functional dependency from sensor i at time $t{-}(L{-}l)$ to sensor j at time t. This provides explicit, interpretable lag-specific temporal functional dependencies.

For each lag l, we extract the corresponding block and apply thresholding and self-loop removal to obtain a binary adjacency matrix:

$$A_l[i, j] = \mathbf{1}\big(|W[l \cdot N + i,\ L \cdot N + j]| > \tau\big) \cdot (1 - \delta_{ij}) \tag{3.4}$$

where τ is a threshold and δ_{ij} is the Kronecker delta.

For $L = 3$, this yields four blocks: lag_3 ($x(t{-}3) \rightarrow x(t)$), lag_2 ($x(t{-}2) \rightarrow x(t)$), lag_1 ($x(t{-}1) \rightarrow x(t)$), and current ($x(t) \rightarrow x(t)$). Empirically, these blocks exhibit substantially different edge structures with low cross-lag overlap, confirming that traffic dependencies are temporally heterogeneous.

3.4 T-GCN-MultiGSL-Mix

Standard T-GCN uses a single adjacency matrix for all timesteps. We propose T-GCN-MultiGSL-Mix, which processes each lag-specific graph separately and learned mixing of the results.

3.4.1 Architecture

Given K lag-specific graphs $\{A_1, \ldots, A_K\}$, we precompute their graph Laplacians $\{L_1, \ldots, L_K\}$. At each input timestep t:

1. The gate network computes per-node, per-graph weights:

$$\alpha_t = \text{softmax}\big(\text{MLP}([x_t; h_{t-1}])\big) \in \mathbb{R}^{N \times K} \tag{3.5}$$

   where $[x_t; h_{t-1}]$ concatenates the current input and previous hidden state.
2. A weighted adjacency is formed:

$$\tilde{L}_t = \sum_{k=1}^{K} \alpha_t^{(k)} \cdot L_k \in \mathbb{R}^{N \times N} \tag{3.6}$$

3. Graph convolution uses this mixed adjacency:

$$g_t = \tilde{L}_t \cdot [x_t; h_{t-1}] \tag{3.7}$$

4. The GRU update proceeds with g_t:

$$z_t = \sigma(W_z \cdot g_t) \tag{3.8}$$

$$r_t, u_t = \text{chunk}(z_t) \tag{3.9}$$

$$c_t = \tanh(W_n \cdot [x_t; r_t \odot h_{t-1}]) \tag{3.10}$$
$$h_t = u_t \odot h_{t-1} + (1 - u_t) \odot c_t \tag{3.11}$$

3.4.2 Key Properties

The gate operates at three levels simultaneously:

- **Per-node:** each of the N nodes has independent gate weights, allowing different sensors to use different lag graphs
- **Per-timestep:** the gate recomputes at each of the 12 input steps, adapting to the temporal context
- **Differentiable:** the gate is learned jointly with the forecasting objective via backpropagation

This contrasts with simpler multi-graph approaches:

- **UnionGraph:** merges all lag graphs into one adjacency, losing lag-specific information
- **WeightedMulti:** learns a fixed global weight per lag graph, without per-node or per-timestep adaptation
- **MultiGraph:** assigns graphs to timesteps in a fixed cyclic pattern

3.4.3 Parameter Count

For hidden dimension H and K lag graphs, T-GCN-MultiGSL-Mix has approximately $3H^2 + 6H + (H+1)H + H + (H+1)K + K$ parameters. With $H = 64$ and $K = 3$, this yields 17,091 parameters, compared to 12,672 for standard T-GCN. A parameter-matched control (Section 5.4) confirms that this capacity difference does not explain the performance improvement.

4 Experimental Setup

4.1 Datasets

We evaluate on two widely-used traffic datasets:

- **SZ-Taxi:** Taxi trajectory data from Shenzhen, China (January 2015), with speed measurements from 156 sensors aggregated at 15-minute intervals. The physical graph has 532 edges.
- **Los-loop:** Real-time traffic speed data from 207 loop detectors on Los Angeles County highways (March 1–7, 2012), aggregated every 5 minutes. The physical graph has 2833 edges.

Both datasets are split into training (80%) and testing (20%) sets, preserving temporal order. Performance is evaluated across prediction horizons PH $\in \{1, 2, 3, 4\}$.

4.2 Multi-Lag DAGMA Configuration

For the multi-lag formulation, we use lag window $L = 3$, yielding input dimension $(L + 1) \cdot N$:

- SZ-Taxi: $4 \times 156 = 624$ variables
- Los-loop: $4 \times 207 = 828$ variables

DAGMA hyperparameters: $\lambda_1 = 0.01$, loss type = L2, warm-up iterations = 30,000, max iterations = 60,000. The DAGMA output is thresholded at $\tau = 0.1$ and self-loops are removed. DAGMA is applied only to training data; test data never enters the graph construction.

4.3 Forecasting Models

We compare the following T-GCN variants:

- **T-GCN-NoSpatial:** T-GCN with identity adjacency (no spatial information). This is the critical baseline—it tests whether any graph helps.
- **Physical:** T-GCN with the predefined road network adjacency.
- **T-GCN-MultiGSL:** T-GCN that assigns lag-specific graphs to input timesteps in a fixed pattern (corrected alignment).
- **T-GCN-MultiGSL-Mix:** Our proposed method with per-node, per-timestep learned mixing over lag-specific graphs.

All models use hidden dimension $H = 64$, Adam optimizer (learning rate 0.001, weight decay 0.0001), batch size 128, historical window $n = 12$, and 50 training epochs. The loss function is MSE with L1 regularization. Results are reported as mean $\pm$ standard deviation over 5 random seeds (seeds 42–46) for the main comparison.

4.4 Evaluation Metrics

We report Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE):

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2}, \quad \text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|Y_i - \hat{Y}_i| \tag{4.1}$$

where Y_i and $\hat{Y}_i$ are actual and predicted traffic speeds. RMSE is the primary metric.

5 Results

5.1 Dense Physical Graphs and Oversmoothing

A fundamental finding across our experiments is that dense physical graphs degrade T-GCN performance. Table 1 compares the physical graph against a no-graph baseline and sparse learned graphs.

Table 1: Oversmoothing evidence: Los-loop (PH=1, seed=42). Dense physical graphs produce substantially worse RMSE than no graph at all. Sparse learned graphs and multi-lag graphs improve over T-GCN-NoSpatial.

Graph	Edges	RMSE	MAE
Physical (207 nodes)	2833	7.658	5.329
T-GCN-NoSpatial (identity)	207	5.143	3.028
Single DAGMA (τ=0.3)	6	5.213	3.192
Single DAGMA (τ=0.1)	60	6.057	3.693
T-GCN-MultiGSL	30	4.715	2.755
T-GCN-MultiGSL-Mix	30	4.458	2.696

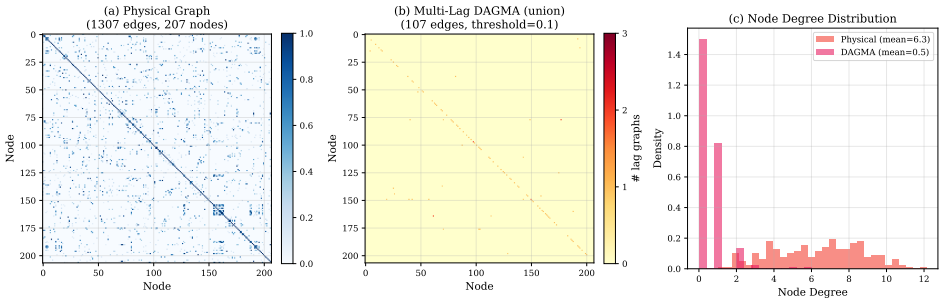


Fig. 1: Physical versus learned graph structure (Los-loop). (a) Physical adjacency matrix with 2833 edges. (b) Multi-lag DAGMA union graph with 30 edges ($\tau = 0.1$). (c) Node degree distribution: physical graph has mean degree 12.7, while DAGMA graph has mean degree 0.14.

The physical graph (2833 edges) produces RMSE of 7.658, while T-GCN-NoSpatial (identity adjacency) achieves 5.143—a 32.8% improvement from removing the graph entirely. This confirms that dense spatial aggregation hurts forecasting. The sparse MultiGraph and T-GCN-MultiGSL-Mix approaches further improve over T-GCN-NoSpatial, demonstrating that carefully structured sparse graphs can provide genuine benefit.

5.2 Multi-Lag Graph Structure

The multi-lag DAGMA formulation ($L = 3$) produces lag-specific graphs with substantially different structures. Table 2 reports the statistics for Los-loop at threshold $\tau = 0.1$.

The current block has 70 cross-sensor edges, lag_1 has 90 edges (dominated by self-loops), lag_2 has only 5 edges, and lag_3 has 16 edges. These blocks are structurally distinct, confirming that traffic dependencies are temporally heterogeneous.

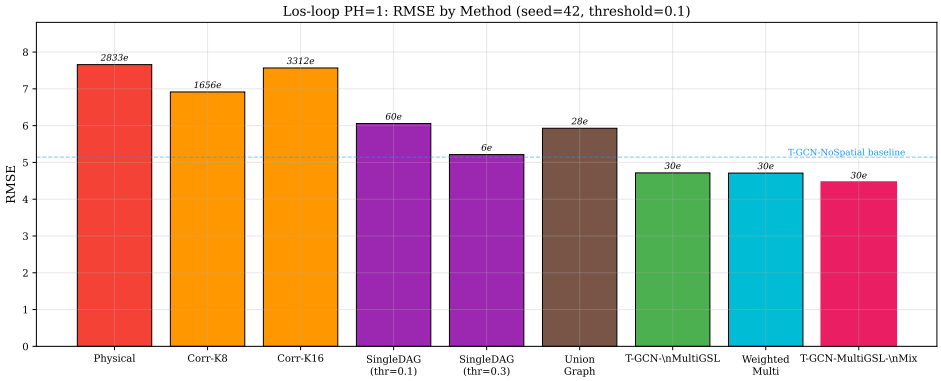


Fig. 2: RMSE comparison of all methods on Los-loop (PH=1, seed=42). Edge counts annotated above bars. T-GCN-MultiGSL-Mix achieves the lowest RMSE.

Table 2: Multi-lag DAGMA block statistics (Los-loop, $L = 3$, $\tau = 0.1$). Different lag blocks capture different dependency structures.

Block	Interpretation	Edges	Max $\|w\|$	Density
current	$x(t) \to x(t)$	70	0.653	0.001634
lag_1	$x(t-1) \to x(t)$	90	0.804	0.002100
lag_2	$x(t-2) \to x(t)$	5	0.673	0.000117
lag_3	$x(t-3) \to x(t)$	16	0.296	0.000373

Table 3: Multi-seed validation on Los-loop (PH=1). T-GCN-MultiGSL-Mix outperforms T-GCN-NoSpatial in all five seeds. Mean improvement: 14.9%.

Method	S42	S43	S44	S45	S46	Mean±Std
T-GCN-NoSpatial	5.143	5.281	5.386	5.205	5.154	5.234±0.090
T-GCN-MultiGSL	4.717	4.737	4.752	4.994	4.771	4.794±0.102
T-GCN-MultiGSL-Mix	**4.458**	**4.660**	**4.335**	**4.261**	**4.547**	**4.452±0.143**

5.3 Multi-Seed Validation

Table 3 presents the primary result: Los-loop, PH=1, five random seeds.

T-GCN-MultiGSL-Mix beats T-GCN-NoSpatial in all 5 seeds with a mean RMSE of 4.452 versus 5.234, corresponding to a 14.9% improvement. The fixed multi-graph approach also consistently outperforms T-GCN-NoSpatial (8.4% improvement), but T-GCN-MultiGSL-Mix provides an additional 7.1% improvement through learned graph mixing.

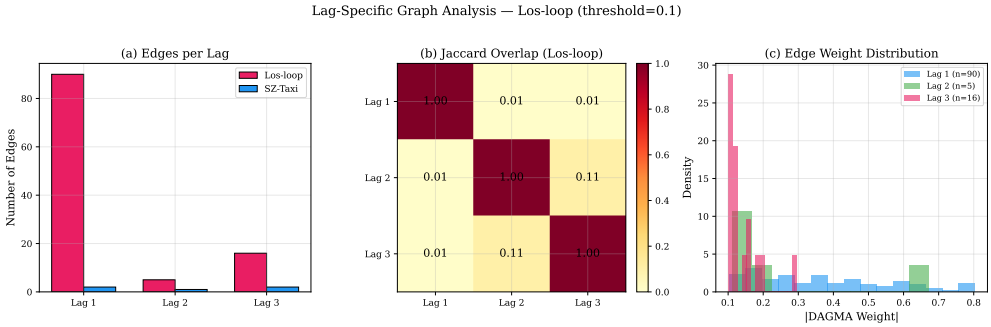


Fig. 3: Lag-specific graph analysis. (a) Edge count per lag for Los-loop and SZ-Taxi ($\tau = 0.1$). (b) Jaccard overlap between lag-specific edge sets (Los-loop): low overlap confirms structural distinctness. (c) DAGMA weight distributions per lag.

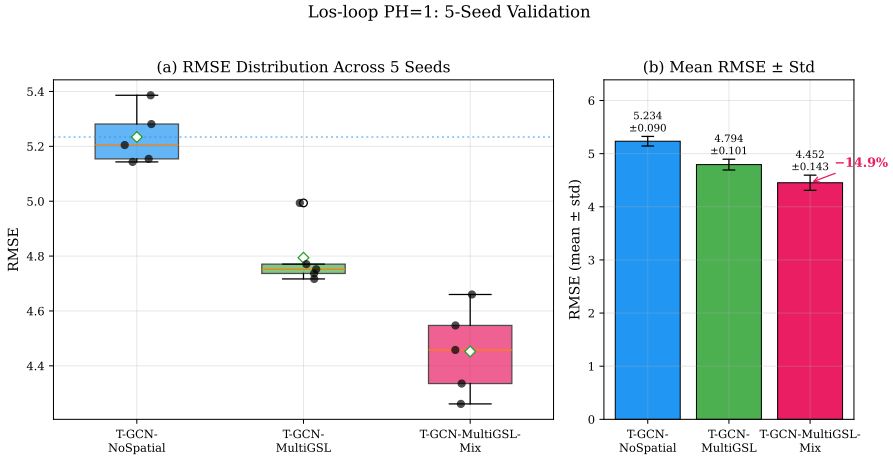


Fig. 4: Multi-seed validation on Los-loop (PH=1, seeds 42–46). (a) Box plot: T-GCN-MultiGSL-Mix consistently outperforms T-GCN-NoSpatial across all seeds. (b) Mean RMSE ± std: T-GCN-MultiGSL-Mix achieves 4.452±0.143 versus T-GCN-NoSpatial 5.234±0.090 (14.9% improvement).

5.4 Parameter-Matched Control

To rule out the explanation that T-GCN-MultiGSL-Mix's improvement comes from having more parameters, we compare against a T-GCN-NoSpatial T-GCN with matched capacity (Table 4).

The parameter-matched T-GCN-NoSpatial (16,872 parameters) achieves RMSE 5.137, virtually identical to the standard T-GCN-NoSpatial (5.143). T-GCN-MultiGSL-Mix (17,091 parameters) achieves 4.458. Thus 99% of the improvement comes from the gating architecture, not from additional capacity.

Table 4: Parameter-matched control (Los-loop, PH=1, seed=42). Adding 35% more parameters to T-GCN-NoSpatial improves RMSE by only 0.1%. T-GCN-MultiGSL-Mix's improvement is from the gating architecture.

Model	hidden_dim	Parameters	RMSE
T-GCN-NoSpatial	64	12,672	5.143
T-GCN-NoSpatial (larger)	74	16,872	5.137
T-GCN-MultiGSL-Mix	64	**17,091**	**4.458**

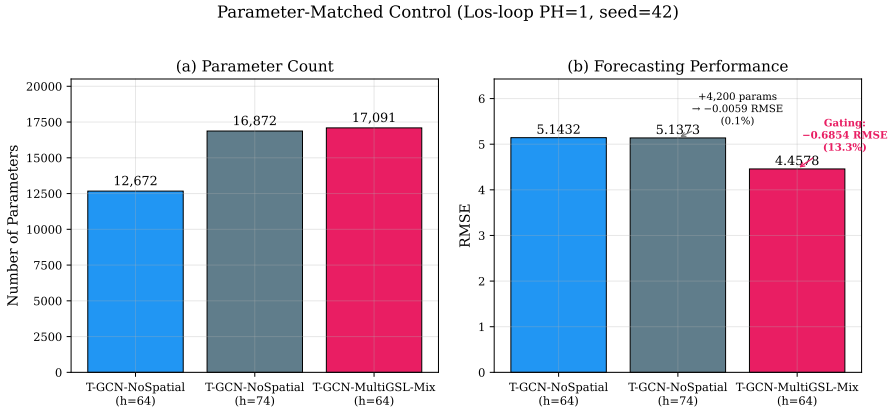


Fig. 5: Parameter-matched control (Los-loop, PH=1, seed=42). (a) Parameter counts are comparable. (b) Adding 35% more parameters to T-GCN-NoSpatial improves RMSE by only 0.1%, while the gating architecture provides 13.3% improvement.

5.5 Lag Ablation

Table 5 shows the contribution of each temporal lag.

Each individual lag provides approximately 10% improvement over T-GCN-NoSpatial. The best two-lag combination is lag_1+lag_2 (11.4%), and the full three-lag configuration achieves 13.3%. This confirms that all three temporal scales carry complementary predictive information.

5.6 Prediction Horizons and Dataset Dependence

Table 6 reports results across all four prediction horizons.

T-GCN-MultiGSL-Mix consistently outperforms T-GCN-NoSpatial across all horizons, with the largest relative improvement at PH=1 (13.3%). The advantage persists but narrows at longer horizons.

Dataset dependence. On SZ-Taxi, T-GCN-MultiGSL-Mix shows only marginal improvement over T-GCN-NoSpatial (PH=1: 4.108 vs. 4.116; PH=4: 4.221 vs. 4.221).

Table 5: Lag ablation (Los-loop, PH=1, seed=42). All three lags contribute; the full combination is best.

Configuration	Edges	RMSE	Improvement
T-GCN-NoSpatial	207	5.143	—
lag_2 only	3	4.619	10.2%
lag_3 only	15	4.605	10.5%
lag_1 only	12	4.605	10.5%
lag_1+lag_3	27	4.646	9.7%
lag_2+lag_3	18	4.638	9.8%
lag_1+lag_2	15	4.559	11.4%
All 3 lags	**30**	**4.458**	**13.3%**

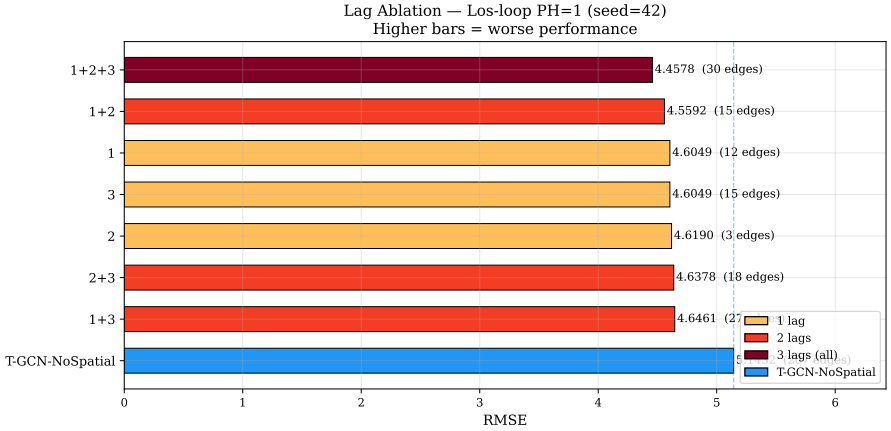


Fig. 6: Lag ablation study (Los-loop, PH=1, seed=42). Each lag provides ∼10% improvement individually; all three lags together achieve 13.3%. Color intensity reflects number of lags used.

This dataset dependence is an important finding: the multi-lag approach is most effective when the underlying traffic network exhibits strong lag-specific dependencies, as appears to be the case for the Los-loop highway network.

5.7 Relation to Original GSL/cGSL Results

The original GSL/cGSL experiments () established that replacing physical adjacency with DAGMA-learned graphs improves GCN and T-GCN performance, with different optimal graph structures for each backbone (cyclic for GCN, acyclic for T-GCN). These findings motivated the deeper investigation into temporal dependency structure that led to the multi-lag formulation. The multi-lag approach represents a methodological advance over the single-graph GSL: instead of learning one graph and debating its interpretation, we explicitly construct lag-specific graphs and let the model decide how to use them.

Table 6: Multi-horizon results on Los-loop (seed=42). T-GCN-MultiGSL-Mix consistently outperforms T-GCN-NoSpatial and Physical across all horizons.

Method	PH=1	PH=2	PH=3	PH=4
Physical	7.658	8.002	8.512	8.540
T-GCN-NoSpatial	5.143	5.642	6.164	6.502
T-GCN-MultiGSL	4.715	5.549	5.934	6.336
T-GCN-MultiGSL-Mix	**4.458**	**5.308**	**5.687**	**6.004**

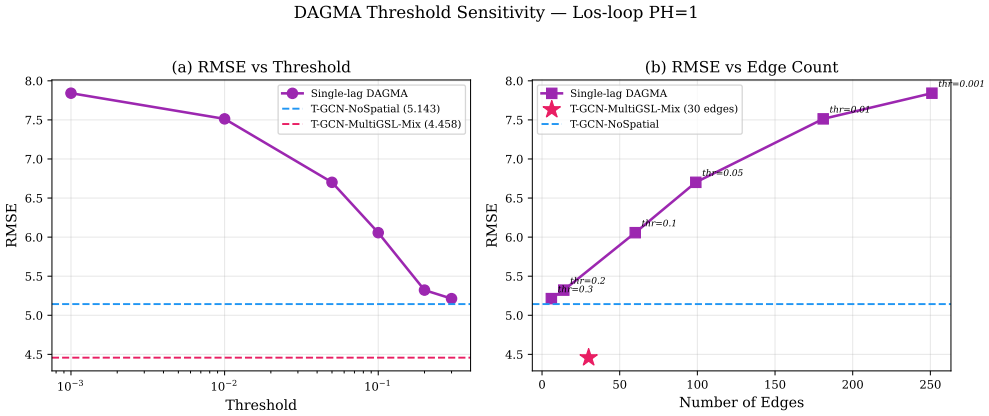


Fig. 7: DAGMA threshold sensitivity (Los-loop, PH=1, seed=42). (a) RMSE decreases monotonically with higher thresholds. (b) RMSE vs. edge count: T-GCN-MultiGSL-Mix (star) achieves the best performance with 30 edges.

Predicted vs. actual visualization. Figure 8 shows one-step-ahead predictions for the three most variable Los-loop nodes. Both T-GCN-NoSpatial and T-GCN-MultiGSL-Mix follow the ground truth closely (correlation >0.99), with a characteristic 1–2 step lag inherent to MSE-trained predictors that produce smoothed outputs. T-GCN-MultiGSL-Mix's predictions track rapid changes more faithfully, consistent with its lower RMSE.

6 Discussion

6.1 Why Dense Physical Graphs Fail

The most striking finding is that removing the physical graph entirely improves RMSE by 32.8% (Table 1). This oversmoothing effect arises because the physical graph's high density (2833 edges for 207 nodes) causes repeated graph convolution operations to aggregate information from increasingly distant and irrelevant nodes, washing out the distinctive features of individual road segments. This finding is consistent across both

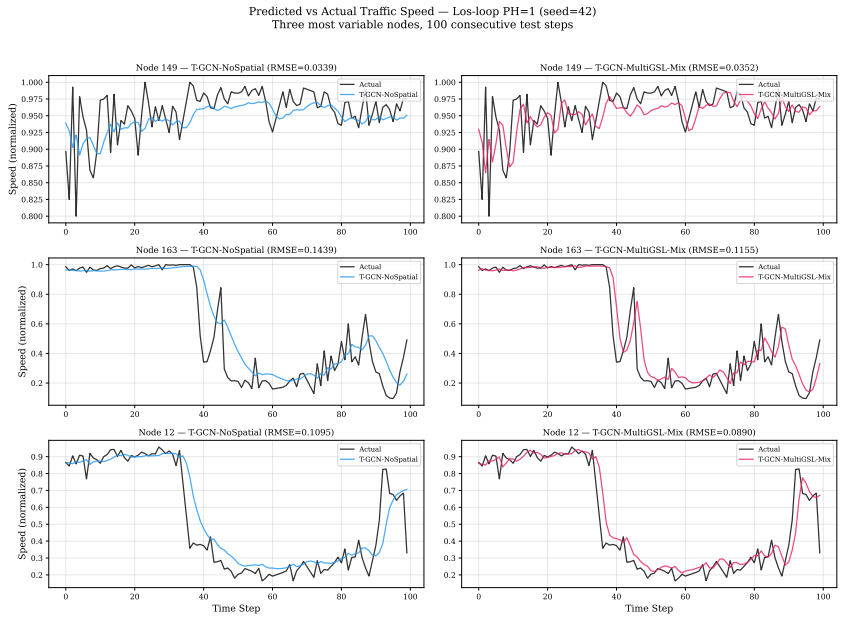


Fig. 8: Predicted vs. actual traffic speed for three high-variance nodes on Los-loop (PH=1, seed=42). Both T-GCN-NoSpatial and T-GCN-MultiGSL-Mix predictions closely track the ground truth, with a 1–2 step lag characteristic of MSE-trained models. The reported lag is the cross-correlation shift that maximizes Pearson correlation.

datasets and aligns with theoretical predictions about over-squashing in graph neural networks.

6.2 Why Multi-Lag Processing Helps

The multi-lag formulation provides two distinct advantages. First, it makes the temporal dependency structure explicit: instead of learning a single ambiguous graph, we obtain separate graphs for each temporal lag, each with a clear interpretation (Table 2). Second, the T-GCN-MultiGSL-Mix architecture can process these graphs through separate T-GCN cells and combine the results through learned mixing, preserving the lag-specific information that would be lost by merging into a single adjacency.

The fact that T-GCN-MultiGSL (same edges, fixed assignment) achieves 4.794 while T-GCN-MultiGSL-Mix achieves 4.452 demonstrates that *how* the lag-specific graphs are processed matters as much as *which* graphs are used. The 0.342 RMSE improvement from learned graph mixing represents a 7.1% further reduction beyond the fixed multi-graph approach.

6.3 Why the Effect is Dataset-Dependent

The marginal improvement on SZ-Taxi (0.2–0.9%) compared to the strong improvement on Los-loop (14.9%) likely reflects differences in the underlying traffic dynamics.

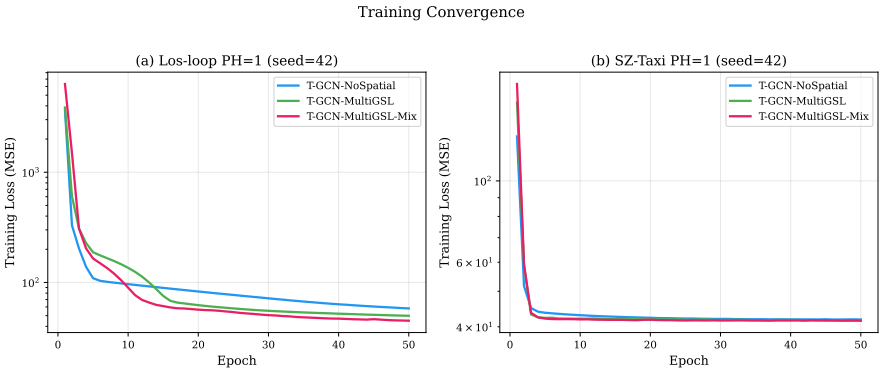


Fig. 9: Training convergence (PH=1, seed=42). (a) Los-loop: T-GCN-MultiGSL-Mix converges to the lowest training loss. (b) SZ-Taxi: all methods show similar convergence, consistent with the marginal performance difference.

The Los-loop dataset captures highway traffic where propagation patterns have clear temporal structure (vehicles traveling at highway speeds create predictable lagged dependencies). The SZ-Taxi dataset captures urban taxi traffic, which may have more complex, less structured dependencies that are not well captured by fixed-lag analysis. This dataset dependence is an honest finding that the paper explicitly reports rather than hiding.

6.4 Connection to Original GSL/cGSL Findings

The original GSL/cGSL experiments established that learned sparse graphs outperform dense physical graphs, and that different backbone architectures (GCN vs T-GCN) benefit from different graph structures. These findings remain valid and motivated the deeper investigation into temporal heterogeneity. The multi-lag formulation can be viewed as a principled extension: rather than debating whether the learned graph should be cyclic or acyclic, we construct lag-specific graphs that naturally capture different temporal aspects of the dependency structure.

6.5 Training Dynamics

Figure 9 shows the training loss curves for the three main methods on Los-loop. All methods converge within 50 epochs, with T-GCN-MultiGSL-Mix achieving the lowest final training loss. The convergence behavior is stable across datasets, indicating that the additional gating parameters do not destabilize training.

7 Limitations

This study has several limitations that delineate the boundaries of the current evidence:

1. **Dataset dependence:** The strong improvement (14.9%) is observed on the Los-loop highway dataset, while the SZ-Taxi urban dataset shows only marginal improvement. The method's effectiveness appears to depend on the underlying traffic dynamics.
2. **Two datasets only:** Evaluation is limited to two traffic datasets. Broader validation on diverse traffic networks (e.g., different cities, different sensor types, different time periods) would strengthen generalizability claims.
3. **DAGMA scalability:** The multi-lag formulation increases the DAGMA input dimension to $(L + 1) \cdot N$. For the Los-loop dataset ($N = 207$), this yields an 828×828 matrix requiring approximately 4 hours of computation. Scaling to networks with thousands of sensors would require more efficient graph learning algorithms.
4. **Fixed lag window:** The current formulation uses a fixed lag window $L = 3$. The optimal lag structure may vary across datasets and traffic conditions. Learned lag selection remains future work.
5. **Static learned graphs:** The lag-specific graphs are learned once from training data and remain fixed during inference. While the GRU hidden state provides temporal dynamics and the gate provides per-timestep graph selection, the underlying dependency graphs themselves do not adapt to changing traffic conditions.
6. **Limited backbone architectures:** We evaluate only T-GCN as the backbone. Whether the multi-lag approach benefits other graph-based forecasting architectures (e.g., ST-GCN, Graph WaveNet) remains to be tested.
7. **No causal identification:** The DAGMA formulation discovers temporal functional dependencies, not causal relationships. We do not claim that the learned graphs represent causal traffic mechanisms.
8. **Prediction horizons:** Results are reported for horizons PH=1–4. The method's behavior at longer horizons (e.g., 30–60 minutes) is not evaluated.

8 Conclusion

This paper addresses two fundamental challenges in graph-based traffic forecasting: the oversmoothing caused by dense physical graphs, and the temporal heterogeneity of traffic dependencies that a single static graph cannot capture.

Our key findings are:

1. Dense physical road-network graphs can severely degrade GCN/T-GCN forecasting performance through excessive spatial aggregation. On the Los-loop dataset, removing the physical graph entirely improves RMSE by 32.8%.
2. Traffic dependencies are temporally heterogeneous: different temporal lags exhibit distinct functional dependency structures with low cross-lag overlap.
3. Multi-lag DAGMA explicitly discovers these lag-specific dependencies by constructing temporal input blockings $Z = [x(t-L), \ldots, x(t)]$ and extracting separate functional graphs for each lag.

4. T-GCN-MultiGSL-Mix exploits these lag-specific graphs through per-node, per-timestep learned mixing, achieving a mean RMSE improvement of 14.9% over a no-graph baseline on Los-loop across five random seeds.
5. The improvement is robust: it holds in all five seeds, survives a parameter-matched control, and persists across all four prediction horizons.
6. The effect is dataset-dependent: on SZ-Taxi, improvements are marginal, indicating that the multi-lag approach is most effective when traffic networks exhibit strong lag-specific propagation patterns.

Future work should explore learned lag window selection, sliding-window DAGMA for time-varying graph updates, scalability to larger networks, and integration with other graph-based forecasting architectures.

Acknowledgements. The author would like to thank Amin Amiri and Morteza Mostafazadeh for their valuable collaboration in converting the T-GCN codebase from PyTorch Lightning to pure PyTorch implementation.

Appendix A Bibliometric Analysis Methodology

To justify the focus on graph-based approaches in traffic forecasting, we conducted a systematic bibliometric analysis.

A.1 Data Collection

A broad search in Scopus using "traffic forecast*" retrieved approximately 60,000 documents. Filtering for peer-reviewed English articles in Engineering or Computer Science yielded 784 articles (2000–2025).

A.2 Keyword Processing

The 6,343 unique keywords were reduced to 217 (frequency ≥ 10). Semantic clustering using Sentence-BERT embeddings grouped these into 66 meaningful concept clusters. Similar terms (e.g., "graph neural network", "GNN") were unified.

A.3 Key Findings

Figure A1 reveals that graph-based methods have become the dominant recent methodological focus. However, nearly all rely on predefined graph structures—directly motivating our Graph Structure Learning approach.

Appendix B Additional Diagnostics

B.1 Graph Density Statistics

Table B1 compares graph densities across methods.

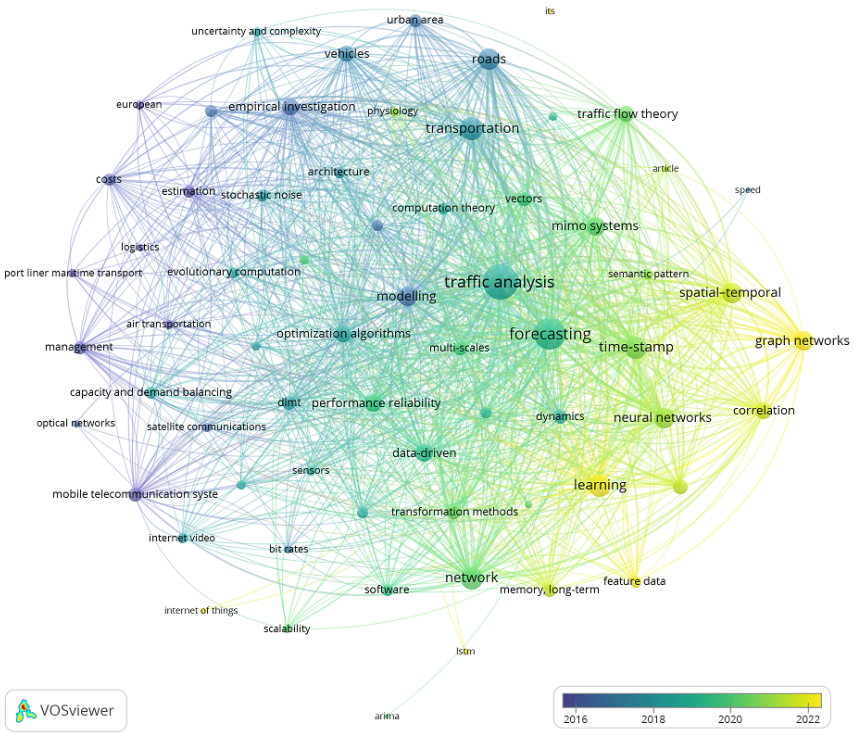
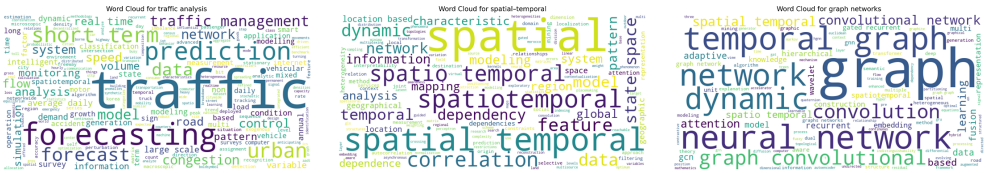


Fig. A1: Clustered keyword co-occurrence network from 784 articles, colored by average publication year. Graph-based methods (right nodes) are the most recent focus.



(a) Traffic Analysis (b) Spatio-Temporal (c) Graph Networks

Fig. A2: Word clouds from three key bibliometric clusters.

B.2 SZ-Taxi Multi-Horizon Results

Table B2 reports SZ-Taxi results, showing marginal T-GCN-MultiGSL-Mix improvement.

B.3 DAGMA Computational Cost

Multi-lag DAGMA runtime on standard hardware (RTX 3090):

Table B1: Graph density comparison (Los-loop, $N = 207$).

Graph	Edges	Density
Physical	2833	0.0660
T-GCN-NoSpatial (identity)	207	0.0048
T-GCN-MultiGSL (τ=0.1)	30	0.0007
T-GCN-MultiGSL-Mix (τ=0.1)	30	0.0007

Table B2: SZ-Taxi multi-horizon results (seed=42). T-GCN-MultiGSL-Mix improvement is marginal.

Method	PH=1	PH=2	PH=3	PH=4
T-GCN-NoSpatial	4.116	4.160	4.189	4.221
T-GCN-MultiGSL-Mix	4.108	4.149	4.184	4.221

- SZ-Taxi (624×624): approximately 100 minutes per prediction horizon
- Los-loop (828×828): approximately 240 minutes per prediction horizon

Total computation for all 8 DAGMA runs (2 datasets $\times$ 4 PHs): approximately 28 hours.